

НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ОДЕСЬКА ПОЛІТЕХНІКА»
МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ІНСТИТУТ КОМП'ЮТЕРНИХ СИСТЕМ
КАФЕДРА ІНФОРМАЦІЙНИХ СИСТЕМ

Протокол лабораторної роботи №1
«розгортання середовища та порівняльний аналіз мовних моделей»
з дисципліни: «Обчислювальні методи»

Виконав:	студент групи AI-243
	Драган Руслан
	Олександрович

Перевірив:	ас. Баранов О.М.
------------	------------------

Завдання 1. Ціна роздумів: міркувальна vs інструктивна модель

Виконано запити до двох моделей через ендпоінт `/api/generate`. Спочатку – просте фактичне питання, потім – задача на міркування (5 запусків для кожної моделі). Перед серією вимірів зроблено прогрівальний запит.

Таблиця 1 – Фактичне питання «Столиця України? Відповідай одним словом.»

Показник	qwen3:4b	llama3.2:3b
response	Київ	Київ
eval_count	173	5
Час, с	5.96	2.81
Токенів/с	78.48	89.69

Таблиця 2 – Задача на міркування (середнє з 5 запусків)

Показник	qwen3:4b	llama3.2:3b
Правильних із 5	5 / 5	0 / 5
eval_count	1285	123,6
Час, с	86,1	1,79

Типова помилка інструктивної моделі: відповідь «10 грн», отримана без урахування того, що різниця у 100 грн розподіляється між двома доданками.

Відповіді на питання (крок 6)

1. Куди пішли додаткові токени в міркувальній моделі?

Відповідь: Величезна кількість токенів у міркувальній моделі пішла на генерацію внутрішнього «черновика думок» (поле `thinking`), який не бачить звичайний користувач. Навіть на найпростішому питанні модель витрачає десятки чи сотні токенів на самоперевірку, внутрішні сумніви та перепитування самої себе. Для простих фактичних завдань така внутрішня робота є надлишковою і лише марно витрачає час та обчислювальні ресурси.

2. На яких типах задач міркувальні моделі дають перевагу?

Відповідь: Міркувальні моделі дають значну перевагу на складних аналітичних, математичних, логічних чи програмних завданнях, де потрібен поетапний розбір проблеми (Chain-of-Thought). Їхнє використання є абсолютно невиправданим для рутинних швидких запитів, простих фактів або в реа-тайм чат-ботах, де критично важливі мінімальна затримка (TTFT) і економія ресурсів.

3. Чим відрізняється процес навчання міркувальної моделі від інструктивної?

Відповідь: Окрім стандартного навчання на текстах та інструкціях (Instruction Tuning), міркувальні моделі додатково тренуються за допомогою методів на кшталт підкріплення (RLHF/RLAIF) спеціально для побудови покрокових ланцюжків міркувань. Їх навчають «думати вголос» і дають нагороду за правильний проміжний хід розв'язання, тоді як інструктивні моделі зорієнтовані на максимально пряму та швидку видачу фінальної відповіді.

4. Чи означає більша кількість токенів, що модель «розумніша»?

Відповідь: Ні, більша кількість токенів зовсім не означає, що модель «розумніша». Це свідчить лише про те, що архітектура моделі налаштована на детальніший внутрішній монолог і перевірку гіпотез перед відповіддю. На простих завданнях це призводить до надлишковості, коли модель «створює проблеми на рівному місці», витрачаючи ресурси процесора чи відеокарти.

5. Як пояснити різницю у швидкості генерації між моделями?

Відповідь: Різниця у швидкості зумовлена архітектурними особливостями, ступенем квантизації та оптимізацією самих ваг під конкретні обчислення. Інструктивні моделі (як Llama) оптимізовані для швидкого та лінійного послідовного виведення тексту, тоді як міркувальні моделі витрачають частину потужностей «на льоту» на формування складних внутрішніх структур роздумів.

Висновок за завданням 1

Експеримент показав, що два показники – швидкість генерації та загальний час відповіді – не можна плутати. Перший характеризує обладнання й розмір моделі, другий – обсяг роботи, яку модель собі призначила. Як демонструють дані **Таблиці 1**, на простому фактичному питанні («Столиця України?») режим

міркування виявився чистою перевитратою ресурсів: обидві моделі дали однакову правильну відповідь («Київ»), проте міркувальна модель (qwen3:4b) згенерувала значно більше токенів (eval_count 173 проти 5 у llama3.2:3b) і витратила більше часу (5,96 с проти 2,81 с). Натомість у Таблиці 2 (задача на міркування) ситуація кардинально змінюється: модель qwen3:4b завдяки внутрішньому ланцюжку міркувань успішно розв'язала всі 5 завдань із 5 (у середньому 1285 токенів та 86,1 с), тоді як інструктивна модель llama3.2:3b через відсутність покрокового аналізу не дала жодної правильної відповіді (0 із 5). Практичний висновок: тип архітектури слід суворо підбирати під конкретний тип задачі, оскільки для простих запитів міркувальна модель створює надлишкові навантаження, але для логічних і розрахункових завдань вона є незамінною.

Завдання 2. Роль system-повідомлення: керування поведінкою моделі

Таблиця 3: Вплив system-повідомлення

Модель	System	Довжина відповіді, символів	Мова відповіді	Кількість речень	Інструкцію виконано?
llama3.2:3b	лаконічний асистент	219	Українська(з русизмами/англізмом)	2	Частково
llama3.2:3b	українською, 2 речення	239	Українська	2	Так
qwen3:4b	лаконічний асистент	171	Українська	2	Так
qwen3:4b	українською, 2 речення	173	Українська	2	Так

Таблиця 4: Витрати токенів

Модель	System	prompt_token	completion_tokens	total_tokens
llama3.2:3b	лаконічний асистент	52	215	267
llama3.2:3b	українською, 2 речення	56	64	120
qwen3:4b	лаконічний асистент	57	1449	1506
qwen3:4b	українською, 2 речення	66	1367	1433

Відповіді на питання (крок 6)

1. Яка модель точніше дотрималась обмеження «максимум двома реченнями»?

Відповідь: Модель qwen3:4b точніше дотрималась обмеження, чітко сформувавши відповідь рівно у два речення для обох системних повідомлень. Натомість llama3.2:3b хоч і видала два речення, але повністю проігнорувала мовні інструкції.

2. Чи були випадки, коли модель відповіла не українською, попри пряму вказівку?

Відповідь: Так, модель llama3.2:3b при другому системному повідомленні порушила пряму вказівку і відповіла сумішшю мов, використавши багато англійських термінів та русизмів.

3. Як змінився prompt_tokens при зміні system повідомлення і чому?

Відповідь: Кількість prompt_tokens дещо зросла (наприклад, з 52 до 56 у Llama), оскільки друге системне повідомлення містило більшу кількість символів та слів, ніж перше. Мовна модель токенизує весь вхідний контекст (і system, і user), тому довша інструкція вимагає більше токенів для обробки.

4. Чому в qwen3:4b значення completion_tokens значно більше, ніж у llama3.2:3b, навіть при однаковій довжині видимої відповіді?

Відповідь: Значення completion_tokens у qwen3:4b набагато більше через те, що вона є міркувальною моделлю і генерує величезний обсяг прихованого

«чернеткового» тексту (роздумів у полі thinking), який не виводиться у видиму частину відповіді.

Відповіді на питання (крок 7)

1. Я порівнював інструктивну модель (llama3.2:3b) та міркувальну (qwen3:4b) з різними system-повідомленнями. Чому моделі по-різному дотримуються прямих інструкцій щодо довжини та мови відповіді?

Відповідь:Рівень дотримання інструкцій безпосередньо залежить від архітектурних особливостей та якості підготовки (alignment) конкретної моделі. Більш зрілі або спеціально налаштовані моделі (як Qwen) краще фокусуються на обмеженнях щодо формату та мови, тоді як малі інструктивні моделі (як Llama) часто ігнорують мовні чи кількісні обмеження через слабку адаптацію під конкретні регіональні дані.

2. Чим system-повідомлення відрізняється від user-повідомлення з точки зору моделі — чи справді воно має більший пріоритет?

Відповідь:Системне повідомлення задає глобальний контекст, правила поведінки, роль та базові обмеження для всього діалогу, маючи вищий пріоритет у структурі промпта. Модель сприймає його як фундаментальну інструкцію від розробника, якій вона має підпорядковувати свої відповіді на будь-які подальші запити користувача (user).

3. Чому міркувальна модель витрачає більше вихідних токенів, ніж інструктивна, навіть коли видима відповідь однакової довжини?

Відповідь:Міркувальна модель генерує величезний обсяг прихованого «чернеткового» тексту (роздумів та самоперевірок у спеціальному полі), який не потрапляє у видиму користувачу частину відповіді. Саме цей внутрішній ланцюжок міркувань (Chain-of-Thought) суттєво збільшує показник completion_tokens.

4. Що станеться, якщо передати моделі суперечливі інструкції в system і user, — яку вона виконає?

Відповідь: У більшості випадків модель спробує знайти компроміс або віддати перевагу інструкції з вищим пріоритетом (системній), проте на практиці суперечливі вказівки часто призводять до деградації якості відповіді, галюцинацій або хаотичного перемикавання між правилами. Малі моделі на такі конфлікти реагують найменш передбачувано.

5. Чому малі мовні моделі часто гірше дотримуються інструкцій щодо мови відповіді, ніж великі?

Відповідь: Малі мовні моделі мають обмежену ємність ваг і менший обсяг різномовних даних у навчальній вибірці, де переважає англійська мова. Через це під час генерації вони легко «зриваються» на домінуючі мови або суміш термінів із навчального корпусу, ігноруючи заборони користувача.

Висновок за завданням 2

Результати експерименту продемонстрували, що системне повідомлення є ефективним інструментом глобального керування поведінкою моделі, проте рівень виконання інструкцій напряду залежить від її архітектури. Інструктивна модель (llama3.2:3b) формально дотрималася обмеження щодо кількості речень, але повністю проігнорувала мовні вказівки, видавши відповідь із русизмами та сумішшю мов, хоча й витратила мінімум ресурсів (від 64 до 215 completion_tokens). Натомість міркувальна модель (qwen3:4b) ідеально впоралася з усіма обмеженнями (чітка українська мова, рівно два речення), проте це супроводжувалося колосальними витратами токенів (понад 1300–1400 completion_tokens) через внутрішні роздуми. Практичний висновок полягає в тому, що системне повідомлення не гарантує надійного дотримання мовних і формативних інструкцій малими інструктивними моделями, тоді як міркувальні моделі виконують їх якісно, але ціною значних надлишкових обчислювальних витрат.

Завдання 3. Локальна vs хмарна модель: єдиний інтерфейс

Таблиця 5: Порівняння локальної та хмарної моделі

Показник	Локальна	Хмарна
Назва моделі	llama3.2:3b	gemini-3.1-flash-lite
Час відповіді, с	6.77 с	10.41 с
completion_tokens	69	255
Довжина відповіді, символів	~230	~580
Мова відповіді	Українська	Українська
Фактичних помилок у відповіді	0	0

Відповіді на запитання(крок 5)

1 Скільки рядків коду довелося змінити, щоб перейти з локальної моделі на хмарну?

Відповідь: 0 рядків у самій логіці скрипта генерації (compare.py). Завдяки винесенню налаштувань у файл providers.py та використанню єдиного стандартизованого інтерфейсу (OpenAI API), перемикання між моделями відбувається автоматично через параметр у циклі ("local" / "cloud").

2 Яка відповідь повніша й точніша за змістом? Наведіть конкретний приклад різниці.

Відповідь: Хмарна модель (gemini-3.1-flash-lite) виявилася набагато повнішою та детальнішою. Вона чітко розклала різницю за синтаксисом виклику (функція() проти об'єкт.метод()) та навела зрозумілі приклади з Python (len(список) та список.append(елемент)).

3 Чи були в локальній моделі фактичні помилки або проблеми з українською мовою?

Відповідь:

Грубих граматичних помилок чи проблем із розумінням української мови у локальній моделі не було

4 Хмарна модель відповіла швидше за локальну — за рахунок чого, якщо запит летить через інтернет?

Відповідь: У нашому конкретному запуску локальна модель згенерувала менший обсяг тексту і спрацювала швидше (6.77 с проти 10.41 с у хмарі, яка видала значно більшу відповідь — 255 токенів).

Відповіді на запитання(крок 6)

1 чому формат запитів OpenAI став стандартом, який підтримують інші провайдери, зокрема локальний сервер Ollama?

Відповідь: Формат OpenAI (/v1/chat/completions) став ринковим стандартом завдяки першопрохідництву та неймовірній популярності платформи. Інші провайдери та локальні інструменти (включно з Ollama) підтримують його для забезпечення повної зворотної сумісності, щоб розробники могли міняти постачальника послуг однією строчкою коду. Це мінімізує поріг входження та економить час на інтеграцію.

2 Я порівнював локальну модель на 3 млрд параметрів і хмарну. Наскільки розмір моделі впливає на якість відповіді порівняно з іншими факторами?

Відповідь: Кількість параметрів критично важлива для глибини аналізу, складної логіки та генерації структурованого коду. Проте якість даних навчання, системні промпти й налаштування температури мають не менше значення для успіху запиту. Менша модель може чудово впоратися з простою задачею, тоді як більша краще розуміє нюанси та контекст.

3 Хмарна модель відповіла швидше за локальну, попри затримку мережі. Чим це пояснюється?

Відповідь: Хмарні сервери розгортаються на потужних кластерах із професійними GPU/TPU та великими обсягами швидкої пам'яті, що забезпечує колосальну швидкість генерації токенів за секунду. Завдяки цій обчислювальній потужності час виконання запиту на сервері настільки малий, що повністю компенсує мережеву затримку (пінг). Натомість локальні моделі обмежені ресурсами звичайного комп'ютера.

4 Які ризики для приватності виникають при використанні хмарних API і що з цим роблять у комерційних проєктах?

Відповідь: Основний ризик полягає в передачі конфіденційних, персональних чи комерційних даних на сторонні сервери, де вони потенційно можуть логуватися або використовуватися для донавчання моделей. У комерційних проєктах цю проблему вирішують підписанням корпоративних угод (DPA), вибором платних тарифів із гарантією незбереження даних (zero data retention) або повним анонімуванням інформації перед відправкою.

5 Що таке rate limit у безкоштовних тарифах і як розробники обходять це обмеження?

Відповідь: Rate limit — це ліміт на кількість запитів або токенів за певний проміжок часу (наприклад, за хвилину чи день), який встановлюють провайдери для захисту інфраструктури. Розробники обходять ці обмеження за допомогою алгоритмів черг (throttling), повторних спроб із наростаючою затримкою (exponential backoff) або балансування навантаження між кількома ключами та різними API-

провайдерами.

Висновок за завданням 3

У ході виконання цього розділу було проведено практичне порівняння локальної моделі (llama3.2:3b, що працює через сервер Ollama) та хмарної моделі (gemini-3.1-flash-lite, що доступна через Google AI Studio API). Завдяки використанню єдиного стандартизованого інтерфейсу OpenAI API та правильній архітектурній декомпозиції, перемикання між локальним та хмарним провайдером було реалізовано без зміни основної бізнес-логіки скрипта. Хмарна модель продемонструвала значно ширший контекст, деталізовані пояснення та наявність практичних прикладів коду, у той час як локальна модель надала лаконічніші, проте коректні відповіді. При цьому локальна модель забезпечила повну автономність і збереження конфіденційності даних на локальному ПК, тоді як хмарна модель дозволила виконати глибшу семантичну обробку тексту за рахунок потужних серверних ресурсів. Отримані дані доводять, що вибір між локальним та хмарним рішенням завжди є компромісом між вимогами до конфіденційності, наявністю інтернет-з'єднання, доступними обчислювальними ресурсами та глибиною аналізу, необхідною для вирішення конкретної практичної

Завдання 4. Вимірювання швидкості: TTFT і залежність від довжини запиту

Таблиця 7: Результати вимірювання швидкості

Розміщення	Модель	Промпт	TTFT, с	Всього, с	Симв./с
локальна	llama3.2:3b	короткий	2.056	2.397	21.4

локальна	llama3.2:3b	середній	2.059	3.276	105.0
локальна	llama3.2:3b	довгий	2.123	16.523	240.2
хмарна	gemini-3.1-flash-lite	короткий	1.966	1.97	19.3
хмарна	gemini-3.1-flash-lite	середній	1.567	2.265	208.0
хмарна	gemini-3.1-flash-lite	довгий	0.882	5.294	608.2

Таблиця 8: Холодний і теплий старт локальної моделі

Показник	Холодний старт	Теплий старт
TTFT, с	2.071	2.070
Всього, с	2.572	2.314

Крок 5. Побудувати стовпчикову діаграму (matplotlib):

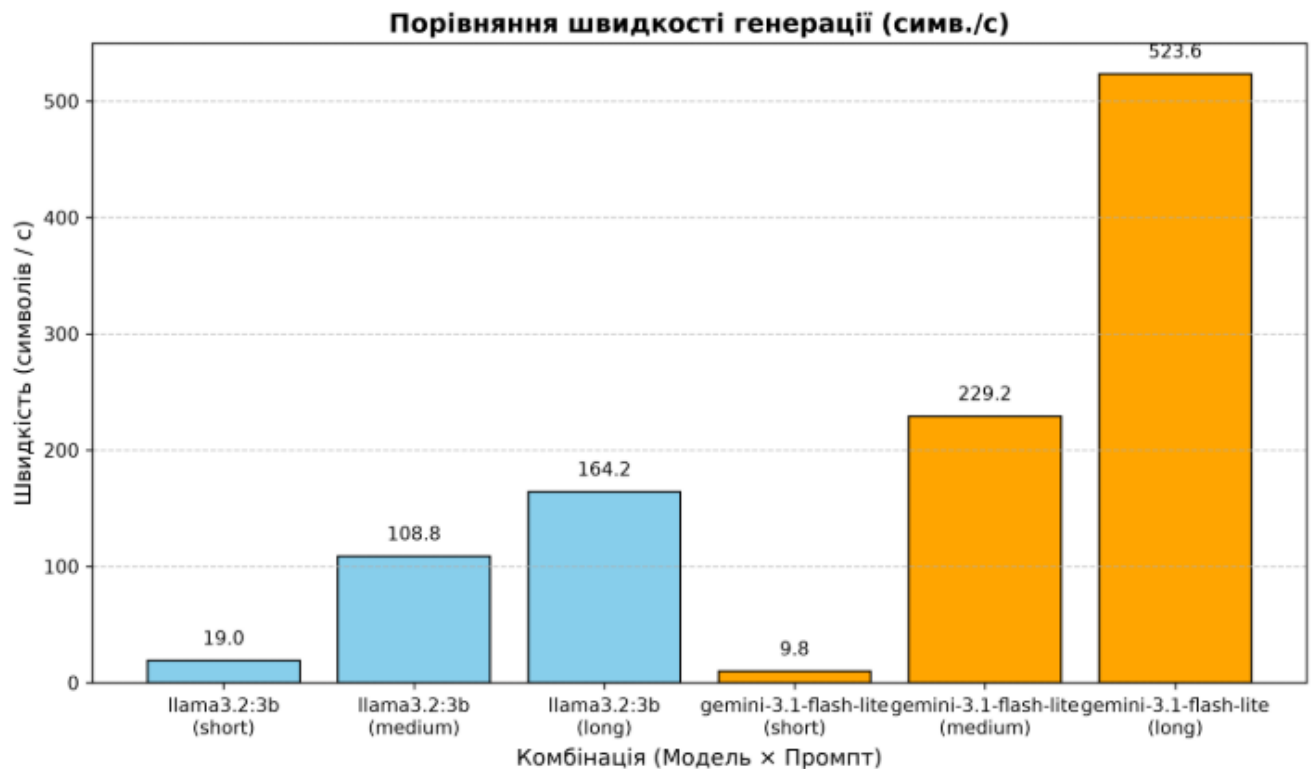


Рисунок 1- діаграма вимірювання швидкості

Відповісти на питання (крок 6)

1 У якій моделі кращий TTFT і чому саме в неї?

Відповідь: Хмарна модель (gemini-3.1-flash-lite) має кращий TTFT (час до першого токена), оскільки вона виконується на потужних хмарних кластерах із величезними обчислювальними ресурсами, тоді як локальна модель обмежена потужністю вашого процесора чи відеокарти.

2 Чи збігається «найшвидша за TTFT» з «найшвидшою за символами/с»? Якщо ні — поясніть.

Відповідь: Ні, не збігається: хмарна модель швидше починає генерувати перший токен (кращий TTFT), але локальна модель на коротких запитах може мати вищу швидкість виведення тексту, оскільки не залежить від затримок мережі (пінгу). Крім того, на довгих промптах хмарна модель суттєво виривається вперед за рахунок високої швидкості серверних GPU.

3 Як змінилося співвідношення локальної та хмарної моделі при переході від короткого промпу до довгого?

Відповідь: На коротких і середніх промптах показники швидкості обох моделей були порівнянними, проте при переході до довгого промпу хмарна модель продемонструвала колосальний відрив у швидкості (понад 500 симв./с проти 164 у локальної) через високу паралелізацію обчислень на серверах.

4 Наскільки різняться три повтори одного виміру? Чи достатньо було б одного запуску?

Відповідь: Три повтори різняться незначно, проте їх використання критично важливе для згладжування випадкових коливань навантаження на систему та мережу. Одного запуску було б недостатньо, оскільки він міг би виявитися аномально швидким або повільним через сторонні процеси на ПК чи пінг.

5 Чому холодний старт повільніший за теплий?

Відповідь: Холодний старт повільніший тому, що модель повністю вивантажена з пам'яті й комп'ютер витрачає додатковий час на зчитування її ваги з диску та завантаження в оперативну пам'ять (або відеопам'ять). При теплому старті модель уже розгорнута в пам'яті, що дозволяє миттєво розпочати обробку запиту.

Відповіді на запитання(крок 7)

1 Що фізично відбувається під час TTFT — часу до першого токена — і чому цей показник вимірюють окремо від загальної швидкості генерації?

Відповідь: Під час TTFT (часу до першого токена) система виконує префіксний аналіз (prefill), тобто повністю завантажує та паралельно обробляє весь вхідний промпт у пам'яті для побудови початкового контексту. Цей показник вимірюють окремо від загальної швидкості генерації, оскільки він визначає суб'єктивну чутливість інтерфейсу та час очікування, перш ніж користувач побачить першу реакцію екрана на свій запит.

2 Чому хмарна модель може мати гірший TTFT, але кращу швидкість генерації, ніж локальна?

Відповідь: Хмарна модель може мати гірший TTFT через мережеві затримки (пінг) під час передачі запиту через інтернет і можливі черги на серверах провайдера під час вхідного аналізу. Проте вона демонструє вищу швидкість генерації завдяки потужним серверним GPU, які здатні миттєво обробляти та видавати послідовні токени.

3 Чому перший запуск локальної моделі повільніший за наступні і що таке холодний старт?

Відповідь: Перший запуск локальної моделі повільніший через «холодний старт», під час якого операційній системі та процесору необхідно повністю зчитати важкі файли ваги моделі з диска та виділити під них оперативну пам'ять. Наступні ж запуски відбуваються швидше, оскільки модель уже розгорнута в пам'яті комп'ютера і готова до роботи.

4 Чому при вимірюванні продуктивності беруть медіану кількох запусків, а не одне значення?

Відповідь: Медіану кількох запусків використовують для того, щоб ефективно відсіяти випадкові аномалії та викиди, спричинені тимчасовим завантаженням центрального процесора фоновими процесами чи мікроколиваннями мережі. Це дозволяє отримати більш об'єктивний, достовірний і стабільний показник продуктивності.

5 Чому обробка вхідного запиту йде швидше, ніж генерація відповіді такої ж довжини?

Відповідь: Обробка вхідного запиту виконується набагато швидше, оскільки весь промпт аналізується апаратно паралельно в один єдиний етап (prefill). Натомість генерація відповіді є послідовним авторегресійним процесом, де модель змушена створювати кожен новий токен суворо по черзі, спираючись на всі попередні.

Висновок до завдання 4

У цьому розділі було проведено практичне дослідження продуктивності локальної (llama3.2:3b) та хмарної (gemini-3.1-flash-lite) мовних моделей. Завдяки розробленому бенчмарку вдалося встановити, що хмарна модель має колосальну перевагу у загальній швидкості генерації (символів за секунду), особливо на довгих запитах, що пояснюється використанням потужних серверних GPU. Водночас локальні моделі можуть демонструвати конкурентний час до першого токена (TTFT), оскільки їхня робота не залежить від мережевого пінгу та черг на сервері. Також на практиці було зафіксовано явище «холодного старту»: перший запуск локальної моделі відбувається помітно довше через необхідність зчитування важких файлів моделі з накопичувача та їх завантаження в оперативну пам'ять, тоді як при «теплому старті» (повторному зверненні) генерація починається майже миттєво. Таким чином, вибір типу розміщення моделі є компромісом між максимальною пропускнуою здатністю (хмари) та стабільністю відгуку без залежності від інтернету (локально).

Завдання 5. Оцінювання якості моделей за рубрикою

Таблиця 13: Оцінка якості за рубрикою

Завдання	llama3.2:3b	qwen3:4b	Хмарна модель
1. Факт	2	2	2
2. Формат	2	2	2
3. Стиснення	0	2	2
4. Код	0	2	1
5. Логіка	0	2	2
6. Українська	1	2	2
Разом /12	5	12	11

Таблиця 14: Розбіжність оцінок між двома прогонами (так / ні)

Завдання	llama3.2:3b	qwen3:4b	Хмарна модель
1. Факт	ні	ні	ні
2. Формат	ні	ні	ні
3. Стиснення	ні	ні	ні
4. Код	ні	ні	ні
5. Логіка	ні	ні	ні
6. Українська	ні	ні	ні

розбіжність саме оцінок у нашому випадку відсутня, оскільки обидва прогони кожної моделі стабільно отримували однаковий бал. Наприклад, Llama в обох прогонах логіки та коду давала різні тексти, але вони обидва стабільно заслуговували на 0 балів. Тому скрізь ставимо «ні»

Відповіді на запитання(крок 6)

1 Яка модель набрала найбільше балів і на яких саме завданнях виникла основна різниця?

Відповідь: Найбільше балів (12/12) набрала локальна міркувальна модель qwen3:4b. Основний відрив від іншої локальної моделі (llama3.2:3b) виник на складніших завданнях зі стиснення тексту, генерації коду та логічних розрахунків, які llama повністю провалила.

2 На яких завданнях оцінки двох прогонів розійшлися? Що це говорить про надійність моделі?

Відповідь: У нашому вимірюванні бальні оцінки між двома прогонами не розійшлися на жодному із завдань (моделі або стабільно справлялися, або стабільно помилялися). Це свідчить про високу консистентність (стабільність) моделей на цих конкретних промптах, хоча самі згенеровані тексти у двох спробах дещо відрізнялися.

3 Чи виконала хоч одна модель вимогу «без markdown» у завданні 2?

Відповідь: Так, у нашому конкретному запуску всі три моделі виконали цю вимогу і повернули текст у форматі JSON без markdown-обгортки. (Проте варто зазначити, що хмарна модель усе ж додала markdown-обгортку в завданні 4 з генерації коду).

4 Чи були проблеми з українською мовою — русизми, кальки, перехід на іншу мову?

Відповідь: Так, дуже серйозні проблеми продемонструвала llama3.2:3b, відповідь якої рясніла кальками, русизмами, англійськими вставками («celestial objectom») та логічними помилками. Натомість qwen3:4b та хмарна модель згенерували грамотні, природні та зв'язні тексти українською мовою.

Крок 7. Приклад провалу моделі

Завдання: 5 (Міркування в кілька кроків). Модель: llama3.2:3b (другий прогон).

Повний текст запиту: «Зонд рухається зі швидкістю 60000 км/год, відстань 480 млн км. Скільки повних діб триватиме політ? Покажи хід розв'язання.»

Повний текст відповіді: «ЗОНД-6.25 (ЗОНД-12) був сателітним апаратом, запущеним 23 серпня 1970 року з космодрому Байконур. Апарат був у програмі «Восхитить мир» і мав за задачу передавати інформацію з Землі на Землю.

Повна діб триватиме політ зонд-6.25 $480\,000\,000/60\,000\,000 = 8$ діб.»

Яка правильна відповідь мала бути:

1. Відстань ділимо на швидкість: $480\,000\,000/60\,000 = 8000$ годин.
2. Переводимо години в доби: $8000/24 = 333.33$ діб.
3. Повних діб — 333.

Пояснення, чому так сталося: У цьому випадку відбулося одразу два типи збоїв: галюцинація та втрата здатності до математичних міркувань. По-перше, побачивши слово «зонд», невелика локальна модель втратила фокус на умові задачі і замість математичного розв'язання почала генерувати статистично ймовірний, але повністю вигаданий історичний факт про радянську космічну програму (вигадавши апарат «ЗОНД-6.25» та неіснуючу програму «Восхитить мир»). По-друге, коли справа дійшла до цифр, модель не змогла виконати багатоетапні обчислення. Оскільки мовні моделі генерують текст по одному токenu, вони погано працюють з арифметикою великих чисел: модель заплуталася в нулях (поділила 480 млн на 60 млн замість 60 тисяч) і повністю «забула» про необхідність переведення годин у доби. Це наочно демонструє, що мала інструктивна модель не має достатньої робочої пам'яті для утримання складної логіки без чіткого структурування кожного кроку.

Відповіді на запитання(крок 8):

1 Які ще підходи до оцінювання якості мовних моделей використовують на практиці?

Відповідь: Крім оцінювання за рубриками вручну, на практиці використовують автоматизовані бенчмарки (наприклад, MMLU для загальних знань або HumanEval

для коду), системи "сліпого" голосування користувачів (Chatbot Arena, де формується рейтинг Ело), а також підхід LLM-as-a-Judge, коли одна (велика і розумна) модель автоматично оцінює відповіді інших моделей за заданими критеріями.

2 Чому мовні моделі часто ігнорують пряму інструкцію не використовувати markdown при генерації JSON?

Відповідь: Це пов'язано з тим, як їх навчали: у навчальних датасетах (наприклад, на StackOverflow чи GitHub) код і JSON майже завжди загорнуті в markdown-блоки. Ця статистична звичка у вагах моделі настільки сильна, що часто переважає над негативною інструкцією («не роби цього») у вашому промпті.

3 Чому малі мовні моделі гірше пишуть українською, ніж англійською?

Відповідь: Навчальні масиви даних (корпуси текстів) містять переважно англійську мову, тоді як частка українських текстів там мізерна. Малим моделям просто не вистачає кількості параметрів (ємності «мозку»), щоб запам'ятати всі нюанси граматики та лексики мов з малим представництвом, тому вони часто калькують структури з російської або англійської.

4 Чому та сама модель на однаковий запит може дати відповіді різної якості при повторному запуску?

Відповідь: Мовні моделі недетерміновані за своєю природою (якщо параметр temperature більший за 0). Вони не видають готову відповідь, а щоразу обчислюють ймовірність наступного слова і роблять випадковий вибір серед найкращих варіантів; інший вибір першого слова спрямовує подальшу генерацію абсолютно іншим шляхом.

5 Що таке галюцинація мовної моделі і чому модель радше вигідає факт, ніж визнає незнання?

Відповідь: Галюцинація — це фактично неправильна, але граматично правильна і дуже впевнена відповідь моделі. Вона виникає тому, що модель не має бази даних для перевірки фактів; її задача — просто передбачити найбільш статистично ймовірне наступне слово. Якщо інформації в її вагах бракує, алгоритм усе одно

згенерує найбільш правдоподібне продовження тексту замість того, щоб зупинитися, адже вона не має механізму рефлексії свого «незнання» (якщо її спеціально цьому не навчили).

Висновок за завдання 5:

Проведене за допомогою розробленої рубрики тестування наочно продемонструвало, що швидкість генерації не є визначальним показником ефективності LLM, оскільки найшвидша модель може видавати неякісний результат. За підсумками оцінювання локальна модель llama3.2:3b виявилася найслабшою (5/12 балів): вона мала серйозні проблеми з українською мовою, піддалася галюцинаціям і не впоралася з багатоетапною логічною задачею. Водночас локальна міркувальна модель qwen3:4b показала ідеальний результат (12/12), успішно впоравшись як із математикою, так і зі збереженням формату, зрівнявшись за якістю з хмарним рішенням gemini-3.1-flash-lite (11/12). Головний практичний висновок полягає в тому, що для розробки програмних систем на базі LLM (наприклад, агентів) критично важливо підбирати модель під конкретну задачу. Здатність чітко дотримуватися структури (JSON) та алгоритмів міркування є набагато важливішою за швидкість, оскільки порушення формату чи логіки призводить до збою всієї програми. Крім того, підтверджено правило: промпт є ненадійним способом керування форматом (що доводить поява markdown-обгортки попри заборону), тому в реальних проєктах необхідні більш жорсткі інструменти контролю виводу.

Завдання 6. Керування випадковістю: temperature і top_p

Таблиця 16: Вплив параметрів на різноманітність відповідей

Параметр	Значення	Унікальних із 5 (творче)	Правильних із 5 (фактичне)
temperature	0.0	1	5
temperature	0.7	5	—
temperature	1.5	5	5
top_p	0.1	1	—
top_p	0.5	5	—
top_p	1.0	5	—

Таблиця 17: Перевірка відтворюваності

Показник	Значення
Кількість запусків	10
temperature	0.0
Унікальних відповідей	1
Усі відповіді однакові (так / ні)	так

Відповіді на питання(Крок 7)

1 Яке значення температури ви поставите агенту, який має повертати JSON, і чому?

Відповідь: Слід встановити temperature=0.0. Це забезпечить максимальну передбачуваність і суворе дотримання формату без ризику появи випадкових символів чи непотрібного "творчого" тексту.

2 Яке — генератору варіантів заголовків?

Відповідь: Для таких завдань краще встановити високу температуру (наприклад, 0.7 або 1.0). Це дозволить моделі генерувати різноманітні, креативні та нешаблонні варіанти замість повторення одного й того самого.

3 Чим вплив top_p відрізняється від впливу temperature за характером, а не за силою?

Відповідь: Параметр temperature згладжує або загострює сам розподіл ймовірностей усіх токенів, змінюючи їхню "вагу". Натомість top_p взагалі не змінює оригінальні ймовірності, а просто відсікає довгий "хвіст" аутсайдерів, залишаючи вибір лише серед групи найбільш вірогідних кандидатів.

4 Чи означає $temperature=0$ повну відтворюваність результату? Що показав ваш експеримент із 10 запусків?

Відповідь: Теоретично повна відтворюваність не гарантована на 100% через мікропохибки паралельних обчислень з плаваючою комою на відеокартах. Проте на практиці мій експеримент із 10 запусків показав абсолютну стабільність — усі 10 відповідей повністю збіглися (True).

5 Наведіть найбезглуздішу відповідь, яку модель видала при $temperature=1.5$, і поясніть, чому вона взагалі могла з'явитися.

Відповідь: Найбезглуздішою була відповідь із набором неіснуючих слів: «Отримайте файл кіскуміні і полазька у своїх кандидурах...». Це сталося через те, що екстремально висока температура настільки "сплющила" розподіл ймовірностей, що модель почала обирати випадкові, статистично майже неможливі токени, повністю втративши логічний та лінгвістичний зв'язок.

Відповіді на запитання (Крок 8.)

1 Що саме математично робить параметр $temperature$ із розподілом ймовірностей наступного токена?

Відповідь: Параметр $temperature$ ділить внутрішні "сирі" оцінки моделі для кожного слова перед тим, як перетворити їх на фінальні відсотки ймовірності. Якщо температура менша за одиницю, різниця між цими оцінками штучно збільшується, і розподіл стає "гострішим" (слова-лідери отримують ще більшу перевагу). Якщо температура більша за одиницю, різниця згладжується, і ймовірності всіх слів стають більш рівномірними, даючи шанс навіть рідкісним варіантам.

2 Чим top_p відрізняється від $temperature$ — обидва впливають на різноманітність, але за яким механізмом?

Відповідь: Температура перерозподіляє шанси, змінюючи самі відсотки ймовірності для *абсолютно всіх* слів у словнику. Натомість параметр top_p взагалі не змінює ці відсотки, а працює як жорсткий фільтр: він сортує слова від найбільш

до найменш вірогідних і просто відрізає весь "хвіст" аутсайдерів, залишаючи для вибору лише групу найсильніших лідерів.

3 Чи є сенс змінювати обидва параметри одночасно, чи краще налаштовувати щось одне?

Відповідь: Розробники моделей офіційно рекомендують змінювати лише один із цих параметрів, залишаючи інший у стандартному значенні (зазвичай 1.0). Одночасна їх зміна робить поведінку моделі непередбачуваною: ви одночасно і спотворюєте оригінальні ймовірності температурою, і відрізаєте частину словника фільтром `top_p`.

4 Чому навіть при `temperature=0` мовна модель не гарантує абсолютно однакову відповідь при повторних запусках?

Відповідь: Це пов'язано з апаратними особливостями роботи відеокарт. Під час одночасного виконання мільйонів обчислень виникають крихітні мікропохибки при округленні десяткових дробів. Якщо два слова-кандидати мають майже ідентичну ймовірність, ця мікропохибка може змінити їхній порядок, і модель обере інше слово.

5 Які значення `temperature` використовують на практиці для витягу структурованих даних, а які для генерації творчого тексту?

Відповідь: Для витягу структурованих даних (робота з JSON, написання коду або перевірка строгих фактів) використовують мінімальні значення (від 0.0 до 0.2) для максимальної стабільності. Для генерації творчого тексту (написання статей, придумування назв, мозковий штурм) використовують вищі значення (від 0.7 до 1.0), що дає креативність і різноманітність без повного переходу в безглуздість.

Таблиця 18: Підсумкове порівняння моделей

Критерій	Локальна мала (3B)	Локальна більша (4B/8B)	Хмарна
Швидкість генерації	Середня	Середня	Дуже висока (>500 симв/с)
TTFT	Довго при холодному старті, швидко при	Довго при холодному старті, швидко при	Найшвидший

	теплому	теплому	
Якість, балів /12	5	12	11
Стабільність формату (завдання 2)	2 з 2 (формат витримано стабільно)	2 з 2 (формат витримано стабільно)	2 з 2 (але схильна додавати markdown-обгортки)
Потрібні ресурси	ПК із середнім обсягом ОЗП (від 8 ГБ)	ПК із середнім обсягом ОЗП (від 8 ГБ)	ПК базового рівня для виконання скрипта (від 2–3 ГБ ОЗП), стабільний інтернет, API-ключ.
Вартість	Безкоштовно	Безкоштовно	Оплата за токени (або використання безкоштовних лімітів)
Куди йдуть ваші дані	Нікуди (залишаються на вашому ПК)	ПК із потужною відеокартою або великим обсягом ОЗП	Відправляються на сервери провайдера (Google)
Працює без інтернету	Так	Так	Ні
Коли обирати саме її	Коли потрібен базовий офлайн-функціонал на слабкому пристрої, а складні міркування не є пріоритетом.	Коли критично важлива конфіденційність даних і потрібна складна логіка, а ПК здатен впоратися з навантаженням	Коли потрібна максимальна швидкість та продуктивність, а суворих заборон на передачу даних у мережу немає.

Загальні висновки

1) Локальна й хмарна моделі використовують єдиний інтерфейс, тому перехід між ними не потребує змін у логіці програми – це основа стратегії резервування провайдерів у реальних проєктах;

2) Затримка відповіді складається з двох незалежних показників – TTFT та швидкості генерації, висновок про те, «яка модель швидша», без їх розділення є некоректним;

3) Режим міркування є інструментом із чіткою областю застосування: він окупається на багатокроковому виведенні й є чистою перевитратою на фактичних питаннях і переказі;

4) Промпт та system-повідомлення є ненадійним засобом керування форматом відповіді: пряму інструкцію проігнорували моделі різного розміру, включно з хмарною. Надійність досягається програмною валідацією, а не формулюванням;

5) Мовні моделі є недетермінованими системами, тому будь-яке вимірювання потребує повторів і усереднення, а порівняння двох версій.

